

Department of Computer Science



Submitted in part fulfilment for the degree of BSc.

# **Creation and Exploration of Informal Guidelines for Implementing RoboChart Models in ROS 2**

Freya Goodger

Version 1.0, 2025-May

Supervisor: Dr Alvaro Miyazawa

# Contents

|                                               |            |
|-----------------------------------------------|------------|
| <b>Executive Summary</b>                      | <b>v</b>   |
| <b>Statement of Ethics</b>                    | <b>vii</b> |
| <b>1 Introduction</b>                         | <b>1</b>   |
| <b>2 Background</b>                           | <b>2</b>   |
| 2.1 RoboChart . . . . .                       | 2          |
| 2.2 ROS 2 . . . . .                           | 3          |
| 2.3 Other Concepts . . . . .                  | 4          |
| <b>3 Literature Review</b>                    | <b>5</b>   |
| 3.1 Modelling Languages . . . . .             | 5          |
| 3.2 Robotic Middleware . . . . .              | 6          |
| 3.3 Related Works . . . . .                   | 6          |
| <b>4 Methodology and Design</b>               | <b>7</b>   |
| 4.1 Methodology . . . . .                     | 7          |
| 4.2 Success Criteria . . . . .                | 7          |
| 4.3 Design . . . . .                          | 8          |
| <b>5 Implementation</b>                       | <b>9</b>   |
| 5.1 SRanger . . . . .                         | 9          |
| 5.2 Simulating SRanger . . . . .              | 14         |
| 5.3 Initial Guidelines . . . . .              | 16         |
| 5.4 Alpha Algorithm . . . . .                 | 21         |
| 5.5 Final Guidelines . . . . .                | 23         |
| <b>6 Evaluation</b>                           | <b>25</b>  |
| 6.1 Case Studies . . . . .                    | 25         |
| 6.2 Qualitative Analysis of Results . . . . . | 26         |
| 6.3 Success Criteria Review . . . . .         | 27         |
| 6.4 Conclusion . . . . .                      | 27         |
| <b>A Full Guidelines</b>                      | <b>28</b>  |
| <b>B Code Appendix</b>                        | <b>33</b>  |

# List of Figures

|     |                                                |    |
|-----|------------------------------------------------|----|
| 2.1 | SRanger RoboChart Model. . . . .               | 2  |
| 2.2 | Simple rqt graph. . . . .                      | 3  |
| 5.1 | SRanger running in Gazebo. . . . .             | 15 |
| 5.2 | ROS Graph of SRanger Simulation. . . . .       | 15 |
| 5.3 | Overview of RoboChart's Symbols. . . . .       | 16 |
| 5.4 | Alpha Algorithm Controller 'Movement'. . . . . | 22 |
| 6.1 | SRanger >10 mins runtime. . . . .              | 25 |

# List of Listings

|      |                                       |    |
|------|---------------------------------------|----|
| 5.1  | Initial Python Code (Version 1).      | 9  |
| 5.2  | SRanger Enum class.                   | 10 |
| 5.3  | SRanger Match Case statement (empty). | 10 |
| 5.4  | SRanger Constants.                    | 11 |
| 5.5  | SRanger Publishers.                   | 11 |
| 5.6  | SRanger Subscriber.                   | 11 |
| 5.7  | SRanger Callback.                     | 12 |
| 5.8  | SRanger Clock and Timer.              | 12 |
| 5.9  | SRanger Time Since Clock Reset.       | 12 |
| 5.10 | SRanger 'Moving' State.               | 13 |
| 5.11 | SRanger 'Turning' State: During.      | 14 |
| 5.12 | Twist 'msg'.                          | 18 |
| 5.13 | Initial Python Code (Version 2).      | 20 |
| 5.14 | Parameters for multiple robots.       | 23 |
| A.1  | Twist 'msg'.                          | 29 |
| B.1  | SRanger Code.                         | 33 |
| B.2  | Interface Code.                       | 36 |
| B.3  | Alpha Algorithm Movement Controller   | 37 |

# Executive Summary

Robots are increasingly being used in safety-critical applications, so it is more important than ever to ensure they behave correctly and safely. Modelling can be used to reason out and test behaviour before implementing it, which generally leads to better thought-out solutions. This means that robots should be safer and more consistent. An architect wouldn't start making a building before designing it in detail, so why should a roboticist?

This project aims to create a set of guidelines for taking a testable, well-reasoned model and implementing it. During implementation, no details of the model can slip through the cracks or be misunderstood; otherwise, all the effort of creating the model is wasted.

There is an introduction to RoboChart, ROS 2, and other key concepts, followed by an exploration of different modelling languages and middleware, culminating in the decision to use RoboChart and ROS 2. Previous similar work is discussed before heading into planning how the project will be carried out, where it is revealed that an agile development style was employed to create the guidelines.

Seven success criteria are set: "SRanger RoboChart model implemented.", "SRanger runs in a ROS environment.", "Initial Guidelines created.", "Alpha Algorithm RoboChart model implemented.", "Guidelines updated to final version.", "Guidelines are coherent and easy to follow.", "Guidelines cover a good proportion of the RoboChart language". After that, a run-through of key RoboChart concepts takes place, planning how they will fit into a ROS implementation before heading into the meat of the matter. The report goes through the entire process of creating a ROS implementation from the RoboChart model known as 'SRanger'. Once the in-depth exploration of this model and translation has taken place, there will be a quick discussion of simulating the SRanger implementation. Any thoughts and ideas from the process so far have been distilled into a set of guidelines that anyone should be able to follow to create their own ROS implementation from a RoboChart model.

These guidelines are then put to the test by creating a second implementation, this time for the RoboChart model 'Alpha Algorithm'. Any issues or missing pieces are noted along the way, and the guidelines are edited at

## *Executive Summary*

the end of the process.

There is a quick evaluation of the case study implementations themselves, followed by a qualitative analysis of the guidelines, including experiences of how easy it was to follow them.

The Success criteria are evaluated before the project concludes with the main aim completed, all of the success criteria at least mostly completed, and a solid set of guidelines created from it all.

# Statement of Ethics

As this project aims to help make robots safer by providing guidelines for translation from tested models to actual implementation, in extreme cases, if the guidelines are incorrect, damage may be caused to a robot or its surroundings. This, of course, is unlikely to happen. Most people will simulate implementations before using them in real life, and any issue is more likely to make nothing happen than anything dangerous.

The project itself does not include other people in any way and should be free of ethical considerations.

# 1 Introduction

Robots are increasingly being used in safety-critical applications, so it is more important than ever to ensure they behave correctly and safely.

Modelling a robot's behaviour enables us to reason out and test solutions before actually implementing them. With proper use, models should improve the approach quality, allowing for safer, more consistent robots to be created much more easily. There may be issues, however, in taking the model and implementing it faithfully. This could be due to a misunderstanding, a lack of time, or any number of different reasons.

This project aims to create an informal strategy or set of guidelines for translation from a model to an implementation on a robotic framework. The modelling language used is RoboChart. The robotic framework is ROS 2. The version of ROS 2 being used throughout is Humble, and the programming language is Python.

These guidelines are intended to make translating a RoboChart model into a ROS implementation easier and more understandable, ensuring users don't miss any key steps or features of their models.

This report will take you through the process of creating these guidelines: from deciding to use RoboChart and ROS 2, to testing out the guidelines. It will also detail any interesting findings or observations during the process.





## 2 Background

controller interacts with 'FootBot'. 'MovementI' contains two operations, 'move' and 'stop'. 'move' takes two inputs, 'lv' (linear velocity) and 'av' (angular velocity) and controls robot movement. 'stop' stops the robot. 'ObstacleI' contains an event 'obstacle', which is connected from 'FootBot' to 'SMovement' through 'Movement'. When an obstacle is detected by 'FootBot', this is conveyed to 'SMovement', where that information is handled according to the behaviour.

## 2.2 ROS 2

ROS 2 [5] (Robot Operating System 2) is a robotic middleware based around an anonymous publish and subscribe system. This system allows messages to be passed between processes. As such, you can subscribe to messages from the robotic platform, do any processing required, and publish messages back to the robotic platform. You can also set up separate processes with subscribers and publishers as part of the processing. These connections are arranged as a graph with different processes as nodes.

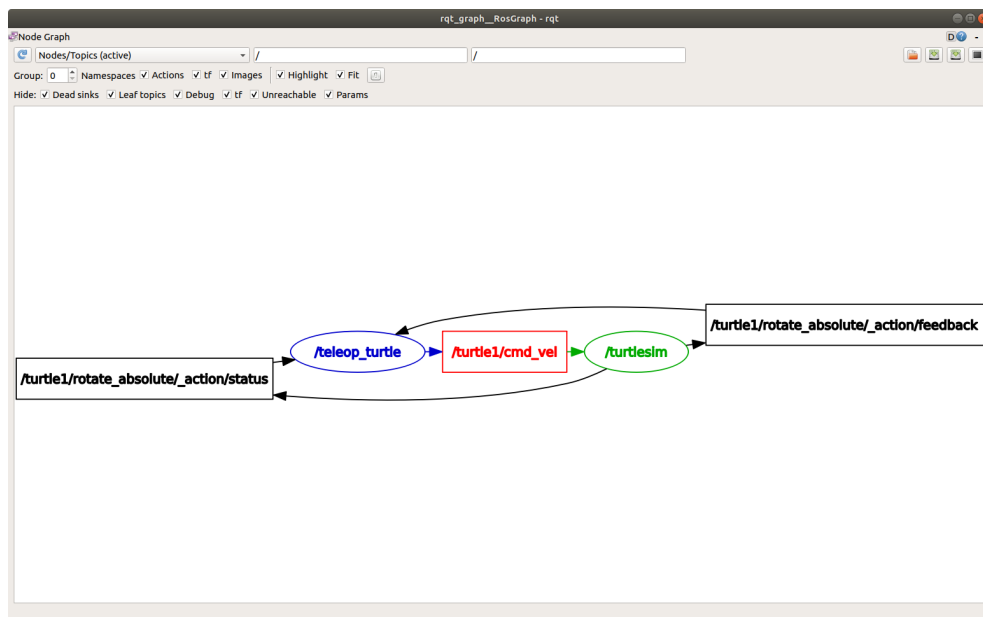


Figure 2.2: Simple rqt graph.

Figure 2.2 shows a simple ROS graph [6], visualised using the rqt\_graph GUI plug-in. It contains two nodes, "/teleop\_turtle" and "/turtlesim". The teleop node is taking any entered keystrokes and publishing them to the "/turtle1/cmd\_vel" topic. These are velocity commands intended for turtle1, the robot being controlled. The 'turtlesim' node is subscribed to this topic

and will act according to its programming when it receives a message (i.e. changing velocity). Two actions outside are sending feedback and status reports from the turtlesim node back to the teleop node so that it knows that the messages are being received and acted upon.

### **2.3 Other Concepts**

**Model** - Representation of an object, person, or system, generally with some abstraction. In this case, we are dealing with models of a robot and its behaviour.

**Domain Specific Language** - A Domain Specific Language, or DSL, is a language made for a specific "domain" or task. Most DSLs discussed in this project will be those designed specifically for robotics and will have limited or no use outside of this domain.

**State Machine** - A State Machine is a collection of states connected by transition conditions that allow movement from one state to another if the condition is met.

## 3 Literature Review

### 3.1 Modelling Languages

Many more general languages are used for behavioural modelling. Chief among them is UML [2], or 'Unified Modeling Language', which has many derivatives. UML is a standardised modelling language which has been managed by the Object Management Group since 1997. It is very popular, but it isn't a one-size-fits-all solution.

Having a language that is exactly tailored to the task at hand can be very useful. It can make things quicker and more efficient, and may provide features that you otherwise wouldn't have. When similar tasks need to be done often, Domain Specific Languages (DSLs) are often created to save future time and energy. SysML [7], or 'Systems Modeling Language', is a DSL made from UML for systems engineering. It can be used for robotics, but even this is not specific enough to the task at hand.

Nordmann et al. [8] suggested that DSLs for robotics are becoming increasingly popular. Many DSLs have been created for robotics, such as RobotML [9], a UML-based DSL that allows automatic generation of platform-independent code. It does, however, seem to be more component-based, where we are more focused on behaviour. Another example is GenoM [10], which allows certain checks, such as deadlock checks, but is an executable language as opposed to RoboChart, which is purely for modelling.

According to Miyazawa et al. [1], to their knowledge, most DSLs for robotics do not have a technique to prove behavioural properties and often have no or limited support to deal with time. RoboChart, having these features, becomes an ideal choice for this project due to being able to prove characteristics of the models being translated and include time-based elements.

## 3.2 Robotic Middleware

Many comparisons on robotic middleware have been done; Elkady and Sobh [11] discuss seventeen different types of robotic middleware in their paper. Some popular options are ROS and RT-middleware.

ROS is open source, can be simulated, and has a substantial number of tools and libraries to do almost anything.

In Macenski et al. [5], some differences between ROS 2 and its predecessor are discussed. Some issues with ROS 1 mentioned are that delivery of data can be inconsistent over lossy connections such as Wi-Fi, that it has a single point of failure, and that it doesn't have built-in security measures. ROS 2 supports Windows and MacOS, as well as Linux, which was the only supported platform for ROS 1. In addition, the last distribution of ROS 1 will also be entering end-of-life on the 31st of May [12], so any guidelines about it would be short-lived. All in all, while some people may still use ROS 1, ROS 2 is an upgrade and will be supported for longer.

There are two stable distributions of ROS 2 currently supported [13], being 'Humble Hawksbill' and 'Jazzy Jalisco'. I decided to use the slightly older 'Humble Hawksbill' as it currently has more third-party support. I have decided to use Python over C++ as there is very little difference, and I have more experience using it.

## 3.3 Related Works

Shaukat et al. [14] apply the RoboChart modelling approach to autonomous firefighting drones. They then put forward a ROS software architecture that aligned with the model, "ensuring the trustworthiness of the system".

Santos et al. [15] created a RoboChart model for an Unmanned Aerial Vehicle. They implemented the state machines using the pytransitions Python library.

A similar student project [16] has been done before. It focused on implementing two RoboChart models in ROS, 'SRanger' and 'Autonomous Chemical Detector'.

# 4 Methodology and Design

## 4.1 Methodology

The first step is to decide on a set of case studies, ideally, this will cover a relatively simple starting example, multiple robots and a concept of time. These will be sourced from the RoboStar website [17].

Once the case studies have been selected, there will be an exploration of each of the RoboChart model elements to find the best way of matching them up with the ROS implementation. Upon identifying the parallels, an agile development style will be employed to create a set of guidelines. The code will be put together for the first example, and an initial set of guidelines will be made in doing so. The code will then be tested and guidelines adjusted if required. These guidelines will then be followed to implement a second, more complex, case study, allowing for more detailed and further-reaching guidelines.

After the guidelines have been created through implementing these two case studies, it will be time for evaluation. First, the case study implementations will be assessed, checking them against the initial models and testing them in different scenarios. The main part of the evaluation will be qualitative analysis of the guidelines themselves, comparing them to other guidelines that exist and measuring how much of the RoboChart language is covered by them, as well as discussing how easy they were to follow when creating the second implementation.

## 4.2 Success Criteria

1. **SRanger RoboChart model implemented.**
2. **SRanger runs in a ROS environment.**
3. **Initial Guidelines created.**
4. **Alpha Algorithm RoboChart model implemented.**

5. Guidelines updated to final version.
6. Guidelines are coherent and easy to follow.
7. Guidelines cover a good proportion of the RoboChart language.

## 4.3 Design

The case studies being used are 'SRanger' [3][4] and 'Alpha Algorithm' [18]. 'SRanger' is a relatively simple starting point to create the basic guidelines from. 'Alpha Algorithm' has parallel state machines, composite states and multiple robots, allowing for more of RoboChart's features to be included in the guidelines.

Referring back to the 'SRanger' model (Figure 2.1) allows us to identify the key features of a RoboChart model. These will be taken in turn, and an equivalent ROS concept will be matched to it.

**Module** - This is the specification of the robotic system in its entirety; it will be the environment where everything is happening.

**Robotic Platform** - The robotic platform would be a ROS node; the solutions these guidelines create should be able to be used with any robotic platform with the same basic capabilities. The robotic platform shall therefore be ignored until testing.

**Controller** - Controllers map to the ROS nodes. They can publish and subscribe to topics. Multiple controllers can be run simultaneously by running multiple nodes.

**State Machine** - State Machines are best implemented as a match case statement contained within a control loop.

**State** - States become the cases in the match case statement. Each state has the code for its functionality and its transition condition.

**Interface** - Interfaces become topics, or sets of topics, when mapped to ROS. These will be implemented as specified in the models. An additional layer, tailoring them to a different robotic platform, will be explained later.

**Operation** - Operations will be the publishers in the code, generally telling the robot what to do.

**Event** - Events will be the subscribers in the code, taking in information from the robot.

# 5 Implementation

## 5.1 SRanger

Now that some concepts have been mapped from RoboChart to ROS, the Python code of the 'SRanger' model (Figure 2.1) can begin development. Firstly, it is important to set up 'main' and the 'Movement' class as seen in Listing 5.1, the 'Movement' class being our controller from the RoboChart model. This is the bare bones version of the code.

Listing 5.1: Initial Python Code (Version 1).

```
1 import sys
2 import rclpy
3 from rclpy.node import Node
4 from rclpy.signal import SignalHandlerOptions
5 from rclpy.executors import ExternalShutdownException,
   MultiThreadedExecutor
6
7 class Movement(Node):
8     def __init__(self):
9         super().__init__('robot_controller')
10
11     def control_loop(self):
12         #Behaviour here
13
14     def destroy_node(self):
15         super().destroy_node()
16
17 def main(args=None):
18     rclpy.init(args = args, signal_handler_options =
        SignalHandlerOptions.NO)
19     node = Movement()
20     executor = MultiThreadedExecutor()
21     executor.add_node(node)
22     try:
23         executor.spin()
24     except KeyboardInterrupt:
25         pass
26     except ExternalShutdownException:
27         sys.exit(1)
```



## 5 Implementation

```
28     finally :
29         node.destroy_node()
30         rclpy.try_shutdown()
31
32 if __name__ == '__main__':
33     main()
```

The 'Movement' class at present has three functions: initialising itself, destroying itself, and an empty control loop. The control loop is where the state machine will be implemented. The code in 'main' sets everything in motion and should not need to be changed.

Next comes the state machine, which requires an Enum class that we will call 'State'; it will list out all the states and assign them numbers from 0. This will be placed above the 'Movement' class. SRanger has two states, 'Moving' and 'Turning'; Listing 5.2 shows the implementation of this. It is important that the import is included and that imports are kept up to date.

Listing 5.2: SRanger Enum class.

```
1 from enum import Enum
2 class State(Enum):
3     MOVING = 0
4     TURNING = 1
```

Now that the states are defined, the 'Movement' class can be assigned one. This can be done by adding "self.state = State.MOVING" to the "\_\_init\_\_" function. The state set at this point should be the initial state (the state that the i in a black circle points to). At this stage, we should also add the match case statement, which will act as the state machine itself (Listing 5.3). This will be implemented within the control loop.

Listing 5.3: SRanger Match Case statement (empty).

```
1 match self.state:
2     case State.MOVING:
3         #Behaviour while in 'Moving' state
4     case State.TURNING:
5         #Behaviour while in 'Turning' state
6     case _:
7         pass
```

Section 3.1 of the RoboChart Reference Manual[19], which goes through the well-formedness conditions of the core language, specifies that a module must contain at least one controller. It also stipulates that a controller must contain at least one state machine, which in turn must contain at least one state. As every RoboChart model must include a state, a genericised version of everything should be added to the initial code to this point. The

## 5 Implementation

full version of the initial code will be specified when the guidelines are formalised.

The next task is to define any constants needed; this should be done below the imports. In this case, the model requires linear and angular velocity constants. These are not defined in the model, so they can be set to whatever fits and changed later if needed. A constant for pi should also be created; import 'math' for this. These can be seen in Listing 5.4.

Listing 5.4: SRanger Constants.

```
1 import math
2 LINEAR_VELOCITY = 0.3 #meters per second
3 ANGULAR_VELOCITY = 1.0 #radians per second
4 PI = math.pi
```

The interfaces must be set up, in this case 'MovementI' and 'ObstacleI'. These are how the 'Movement' node is going to interact with the robot, and they will be used often. These will be written as designed in the model and not tied to a specific robotic platform at this time. 'MovementI' has two operations, 'move' and 'stop'. Of these, only 'move' needs to send a message with contents. The way linear/angular velocities are sent in ROS 2 is with Twist messages, which requires the following line in the import section: "from geometry\_msgs.msg import Twist". In addition, sending an empty message with a stop requires the import "from std\_msgs import Empty". Once the message types have been decided and imported, the publishers (Listing 5.5) can be created in the '\_\_init\_\_' function of the 'Movement' class. Defined in each are: the type of message they publish, the name of the topic they publish to, and the buffer size.

Listing 5.5: SRanger Publishers.

```
1 self.move_publisher = self.create_publisher(Twist, '/move',
2      10)
3 self.stop_publisher = self.create_publisher(Empty, '/stop',
4      10)
```

Next, the attention turns to 'ObstacleI', this requires a subscriber (Listing 5.6) and a callback (Listing 5.7). Defining a subscriber is very similar to the publishers, being located in the same place and having the same components, with one extra component that points to the callback.

Listing 5.6: SRanger Subscriber.

```
1 self.obstacle_subscriber = self.create_subscription(
2     Empty,
3     '/obstacle',
4     self.obstacle_callback,
5     10)
```

## 5 Implementation

Callbacks define how messages are handled when they are received. To ensure decisions on state changes are made within the state machine, a new variable has been created in the '\_\_\_init\_\_\_' function: "self.obstacle\_found = False". This will be set to true by the callback when a message is received. Callbacks are functions of the class and are usually defined between the initialisation and the control loop.

Listing 5.7: SRanger Callback.

```
1 def obstacle_callback(self, msg):  
2     self.obstacle_found = True
```

The 'SRanger' model has timed elements, so a timer and a clock will be implemented. This requires setting up the clock 'MBC' by setting it to zero and creating a timer. The timer period and time unit can be set to whatever feels reasonable, as they are not defined in the model. In this case, a timer period of 0.1 (10Hz) and a time unit of one second are being used. This will all be in the '\_\_\_init\_\_\_' function and can be seen in Listing 5.8.

Listing 5.8: SRanger Clock and Timer.

```
1 self.clock_MBC = 0  
2 self.timer_period = 0.1  
3 self.time_unit = 1  
4 self.timer = self.create_timer(self.timer_period, self.  
    control_loop)
```

To allow the clock to increment, add "self.clock\_MBC += 1" to the start of the control loop. The 'since\_clock\_MBC' function (Listing 5.9) needs to be added to the 'Movement' class to work out the time since the clock was last reset.

Listing 5.9: SRanger Time Since Clock Reset.

```
1 def since_clock_MBC(self):  
2     return self.timer_period * self.clock_MBC / self.  
    time_unit
```

Finally, it is time to define the behaviour of the states. In RoboChart models, states can have up to three actions: one of each 'entry', 'during', and 'exit'. The easiest way to accommodate this is to create an Enum class like the one in Listing 5.2 called 'Action' and containing the three action types. In addition to this, the line "self.action = Action.ENTRY" needs to be added to the '\_\_\_init\_\_\_' function. Each state can now have a match case statement like Listing 5.3, but for these three actions. This should also be included in the initial code.

The entry action for 'Moving' is "move(lv, 0); wait(1)". The implementation for this state can be seen in Listing 5.10. Lines 4 through 7 represent "move(lv,

## 5 Implementation

0)". A new Twist message is created, the linear velocity set to our constant and the angular velocity set to 0 (technically this isn't needed but is included for completeness), the message is then published by 'move\_publisher'. To implement the wait, a new variable was added to the 'Movement' class by adding "self.waiting = 0.1" to the '\_\_\_init\_\_\_' function. It is set to 0.1 to ignore the first clock increase as the first use of the wait will always start between the first and second clock increase, any other time it is used it will be set to the current result of "self.since\_clock\_MBC()". The wait is one time unit long, as RoboChart models are not written in seconds. Any timed elements should be implemented in time units so that the time unit can be changed.

While the model technically doesn't have any during or exit actions in any of the states, it makes sense for the transition conditions to be placed in the during section. When an obstacle has been detected, the subscriber callback will have set 'self.obstacle\_found' to 'True'. '#MBC' calls for a clock reset when an obstacle is detected, so line 12 sees to that. It may feel natural to reset the 'self.obstacle\_found' to 'False'; however, this will just be immediately changed back as the obstacle is still in front of the robot. As there are no exit actions and there is only one transition out of the state, the 'stop(); wait(1)' has been put in the exit section. This allows for an easier wait to be implemented. As 'Turning' has a wait immediately, 'self.waiting' is set. Remember to set the action back to entry before leaving the state.

Listing 5.10: SRanger 'Moving' State.

```
1 case State.MOVING:
2     match self.action:
3         case Action.ENTRY:
4             msg = Twist()
5             msg.linear.x = LINEAR_VELOCITY
6             msg.angular.z = 0.0
7             self.move_publisher.publish(msg)
8             if self.since_clock_MBC() - self.waiting >=
                self.time_unit:
9                 self.action = Action.DURING
10        case Action.DURING:
11            if self.obstacle_found == True:
12                self.clock_MBC = 0
13                self.action = Action.EXIT
14        case Action.EXIT:
15            msg = Empty()
16            self.stop_publisher.publish(msg)
17            if self.since_clock_MBC() >= self.time_unit:
18                self.waiting = self.since_clock_MBC()
19                self.action = Action.ENTRY
20                self.state = State.TURNING
21        case _:
22            pass
```

## 5 Implementation

The 'Turning' state is very similar to the 'Moving' state. The entry section is identical, but with the angular velocity being set instead. The during section can be seen in Listing 5.11. The transition condition can be written as a simple if statement with the reset for the next state inside, which includes resetting the obstacle flag. No exit section is needed for this state.

Listing 5.11: SRanger 'Turning' State: During.

```
1 case Action.DURING:
2     if self.since_clock_MBC() >= PI/ANGULAR_VELOCITY:
3         self.waiting = self.since_clock_MBC()
4         self.obstacle_found = False
5         self.action = Action.ENTRY
6         self.state = State.MOVING
```

Full code for the SRanger implementation can be found in Listing B.1.

## 5.2 Simulating SRanger

To simulate a ROS implementation, it needs to be in the right environment [20]. To simulate 'SRanger', the Virtual Machine provided as part of the AURO module was used, which consists of Ubuntu Linux with ROS 2 and many ROS packages installed, as well as simulation tools like Gazebo [21].

The simulation was done using a TurtleBot3 Waffle Pi, which required an interface layer to be created, as this was not the robotic interface specified in the model. This is just a node with subscribers and publishers converting messages on certain topics to messages on other topics. Listing B.2 contains this code. A package [22] called 'sranger' containing the 'SRanger' code and this interface code was created, allowing the code to be run in the ROS environment.

To run the code, there must be a workspace containing the package. Navigate to the root of that workspace, then, in terminal, type "colcon build --symlink-install". This will build the code. For this simulation, three terminals were used, in each one, the command "source /auro\_ws/install/local\_setup.bash" was run. The first terminal used the 'turtlebot3\_world.launch.py' file to open Gazebo, load in the 'turtlebot3\_world' map and load in the Waffle Pi robot. This was done by running the command "ros2 launch turtlebot3\_gazebo turtlebot3\_world.launch.py". In the second terminal, the interface was set up using the command "ros2 run sranger interface". Finally, in the third terminal, the SRanger controller was run using "ros2 run sranger robot\_controller".

## 5 Implementation

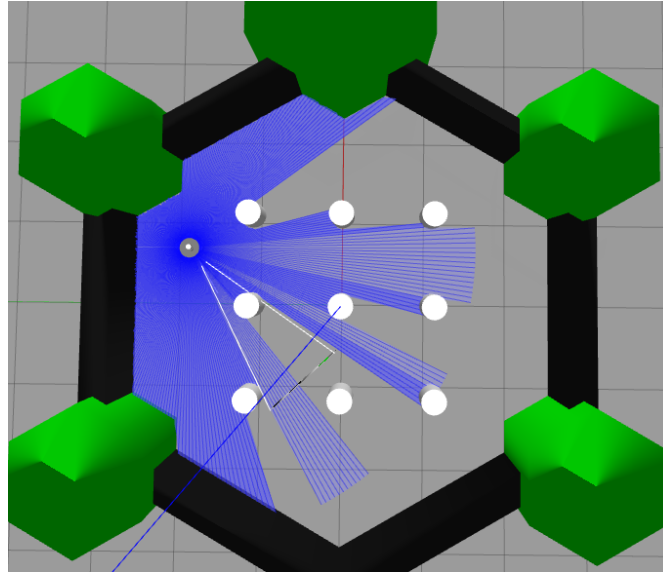


Figure 5.1: SRanger running in Gazebo.

As soon as this final command was run, the robot zoomed off at breakneck pace, too quickly for the simple obstacle detection implemented in the interface code. After tuning the speed down to 0.1 meters per second, the robot worked was moving at a much more reasonable pace and managed to turn as defined in the model, this simulation can be seen in progress in Figure 5.1.

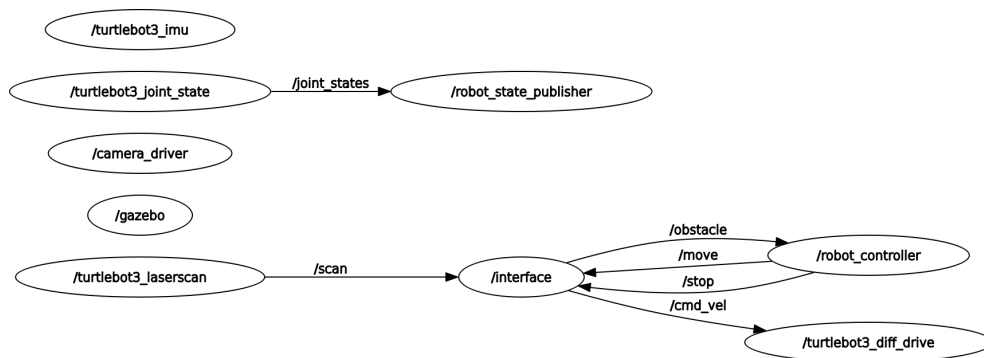


Figure 5.2: ROS Graph of SRanger Simulation.

The ROS graph in Figure 5.2 shows how the '/interface' node allowed the code to be run with a different robotic platform than specified in the model. It takes in laser scan information from '/turtlebot3\_laserscan' through the '/scan' topic and uses that to decide if there is an obstacle; if there is, it sends a message to the controller through '/obstacle'. It also takes in messages from the controller on the '/move' and '/stop' topics, both of which result in a message being sent to '/turtlebot3\_diff\_drive' on the '/cmd\_vel'

topic. The interface is acting as a buffer between the robotic platform and the controller, translating the messages sent to it so the other can understand them. There are also some other nodes in the ROS graph from the simulation environment, but these are not important for this discussion.

### 5.3 Initial Guidelines

A table of the symbols contained within RoboChart has been created to accompany the guidelines.

|                                                                                   |                       |                                                                                   |                        |                                                                                   |                    |                                                                                     |                   |
|-----------------------------------------------------------------------------------|-----------------------|-----------------------------------------------------------------------------------|------------------------|-----------------------------------------------------------------------------------|--------------------|-------------------------------------------------------------------------------------|-------------------|
|  | Module                |  | Import                 |  | Event              |  | Defined Interface |
|  | Controller            |  | Junction               |  | Precondition       |  | Action            |
|  | State Machine         |  | Final State            |  | Postcondition      |  | Primitive Type    |
|  | Interface             |  | Initial State          |  | Constant           |  | Record            |
|  | Connection/Transition |  | Probabilistic Junction |  | Provided Interface |  | Field             |
|  | Robotic Platform      |  | Operation              |  | Required Interface |  | Enumeration       |
|  | Function              |  | Variable               |  | Clock              |  | Literal           |

Figure 5.3: Overview of RoboChart's Symbols.

Following the process of implementing 'SRanger', any thoughts and processes have been distilled into the following guidelines:

#### 1. Set Up

- 1.1 Start with the initial code (Listing 5.13).
- 1.2 Replace instances of `Controller_Name` with the name of your controller, keeping the original capitalisation (lines 16, 18, 43).

#### 2. Constants

- 2.1 Constants should be defined below the imports.
- 2.2 Most constants are not assigned values in models; set them to something and tune them later.

#### 3. Interfaces

- 3.1 Interfaces should be defined in the '`__init__`' function.
- 3.2 Interfaces require a message type.

## 5 Implementation

*3.2a When sending linear and/or angular velocities, this will be 'Twist' from 'geometry\_msgs.msg'.*

*3.2b If all that needs sending is a signal, this will be 'Empty' from 'std\_msgs.msg'.*

*3.2c In certain cases, other message types may be required.*

3.3 Interfaces require a topic to use.

*3.3a This must start with a '/'.*

*3.3b The topic will have the name of the operation/event from the model unless required otherwise.*

3.4 Interfaces require a buffer size.

*3.4a Usually, a buffer size of 10 is more than sufficient.*

3.5 Subscribers require a reference to a callback.

*3.5a The callback will be a function in the controller module.*

*3.5b The callback will define what happens when a message is received.*

*3.5c The callback needs 'self' as a parameter.*

*3.5d The callback needs 'msg' as a parameter.*

*3.5e If the message is empty, the callback may toggle a boolean, allowing processing to happen elsewhere. Ensure this is toggled back at an appropriate point.*

*3.5f If the message has contents, the callback may facilitate storing or making decisions based on them.*

3.6 Subscribers are written in the form "self.subscription\_name = self.create\_subscription(Message\_type, '/topic\_name', self.callback\_name, int buffer\_size)".

3.7 Publishers are written in the form "self.publisher\_name = self.create\_publisher(Message\_type, '/topic\_name', int buffer\_size)".

3.8 When publishing a message is required, include the line "self.publisher\_name.publish()".

*3.8a You must create a 'msg' variable that will be passed as a*



## 5 Implementation

*parameter (even if it's an empty message).*

*3.8b If the message has contents, you must populate it; an example of a Twist 'msg' creation is in Listing 5.12. Twist components must be floats.*

Listing 5.12: Twist 'msg'.

```
1 msg = Twist()  
2 msg.linear.x = LINEAR_VELOCITY  
3 msg.angular.z = ANGULAR_VELOCITY
```

### 4. Timed elements

4.1 If there is a clock defined in your model, you will need to use timed elements.

*4.1a All of these elements are part of the class of your controller.*

4.2 RoboChart models are written in time units.

*4.2a This should be defined and can be changed later.*

*4.2b 1 time unit to 1 second is usually a good starting point.*

*4.2c All timed elements should be written in time units to allow easy changes later.*

4.3 Timed elements require a timer.

*4.3a Timers provide a consistent pulse activating something, in this case, the control loop.*

*4.3b Timers require a timer period, which will decide the frequency. e.g. timer period of 0.1 = 10Hz.*

*4.3c Timers require a function to execute, in this case, the control loop.*

4.4 Timed elements require a clock.

*4.4a This will share a name with the clock in the model.*

*4.4b The clock will start at 0 and should be incremented every time the control loop is pulsed by the timer.*

*4.4c A '#' preceding the clock's name in a model means the clock is reset at that point; this can be done by setting the clock to 0.*

## 5 Implementation

4.4d *In the model, `since(clock_name)` asks for the time since the clock was last reset, i.e. the time on the clock. This is done with the calculation `"self.timer_period * self.clock_name / self.time_unit"`. It may be useful to make this a function.*

4.5 Timed elements allow you to implement waits.

4.5a *If the model includes waits, create a variable in the controller's class for saving the time on the clock.*

4.5b *First, save the time on the clock (Guideline 4.4d) before you transition to another action/state that requires a wait.*

4.5c *Then create an if statement where the wait is that takes the saved time away from the current clock time and compares it to the number of time units the wait is for. i.e. `"current - saved >= x(time_unit)"`.*

## 5. State Machines

5.1 Create a list of all the states in the state machine you are implementing.

5.1a *To match more closely with the RoboChart model, include the initial state.*

5.2 Each state should be added to the Enum class 'State', with the initial state being set to 0 and every subsequent state added being 1 higher.

5.3 Each state should be added to the match case statement.

5.4 Ensure 'self.state' is set to either the initial state if included, or the state it points to.

5.5 If included, the initial state should reset any clocks in use and then transition to the next state.

## 6. State Behaviour

6.1 States have three action types: 'entry', 'during', and 'exit'.

6.1a *While not 'during' actions, transition conditions take place at the same time and so should be implemented in the 'during' section.*

6.1b *If a state has only one transition out of it, it may be convenient to put the consequences of the transition condition*

## 5 Implementation

*in the 'exit' section, despite it not technically being an 'exit' action.*

6.2 Most behaviour will be either using previously set up clocks, interfaces, etc., performing calculations, or both.

6.3 Change state using "self.state = State.STATE\_NAME".

6.4 Change action using "self.action = Action.ACTION\_NAME".

6.5 Before changing states, ensure 'self.action' is also set to 'State.ENTRY'.

### **7. Controllers should send a stop command of some kind when the node is destroyed.**

Listing 5.13: Initial Python Code (Version 2).

```
1 import sys
2 import rclpy
3 from rclpy.node import Node
4 from rclpy.signal import SignalHandlerOptions
5 from rclpy.executors import ExternalShutdownException,
6   MultiThreadedExecutor
7 from enum import Enum
8
9 class State(Enum):
10     STATE_NAME = 0
11
12 class Action(Enum):
13     ENTRY = 0
14     DURING = 1
15     EXIT = 2
16
17 class Controller_Name(Node):
18     def __init__(self):
19         super().__init__('controller_name')
20
21         self.state = State.STATE_NAME
22         self.action = Action.ENTRY
23
24     def control_loop(self):
25         match self.state:
26             case State.STATE_NAME:
27                 match self.action:
28                     case Action.ENTRY:
29                         #Entry action
30                     case Action.DURING:
31                         #During action
```

## 5 Implementation

```
31         case Action.EXIT:
32             #Exit action
33         case _:
34             pass
35     case _:
36         pass
37
38     def destroy_node(self):
39         super().destroy_node()
40
41 def main(args=None):
42     rclpy.init(args = args, signal_handler_options =
43               SignalHandlerOptions.NO)
44     node = Controller_Name()
45     executor = MultiThreadedExecutor()
46     executor.add_node(node)
47     try:
48         executor.spin()
49     except KeyboardInterrupt:
50         pass
51     except ExternalShutdownException:
52         sys.exit(1)
53     finally:
54         node.destroy_node()
55         rclpy.try_shutdown()
56
57 if __name__ == '__main__':
58     main()
```

### 5.4 Alpha Algorithm

This is a swarming algorithm, meaning it is designed for a large group of simple robots. Each robot broadcasts a message, waits for answers, and estimates the number of neighbours. If the number of neighbours is greater than a constant 'alpha', the robot does a random turn. If the number is less than or equal to 'alpha', the robot turns around and heads back to where the swarm presumably is.

The Alpha Algorithm RoboChart model contains many controllers, they will be taken one at a time and implemented. However, this runs into the first error in the guidelines, "Start with the initial code". It does not say what portion of a model this initial code is for, which is fine when there is only one controller, but is not in this instance.

## 5 Implementation

There are multiple versions of the Alpha Algorithm RoboChart model; this section will examine the implementation of revision 1. This is seemingly the most complete version of the model. To avoid clutter, only the images containing the two controllers will be included here; for the entire model, see the RoboStar website [18]. For each controller, the guidelines will be followed until there is something to discuss.

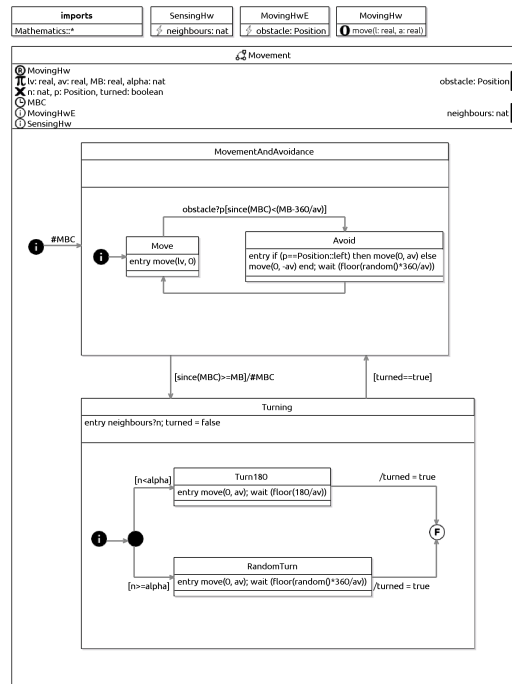


Figure 5.4: Alpha Algorithm Controller 'Movement'.

The first controller can be seen in Figure 5.4. It calls for four constants:  $lv$ ,  $av$ ,  $MB$ , and  $\alpha$ . Of these, the first two are fairly standard and have been discussed.  $MB$  appears to be the time between checking the number of neighbours, and  $\alpha$  is the number of neighbours it is checking for. As Guideline 2.2 states, set them to something and tune them later.  $MB$  needs to be more than  $360/av$ . Therefore, a seemingly reasonable first set of values for these constants is 0.3, 1.0, 361.0 and 3.

The guidelines do not mention variables; the three variables defined in the model should be declared in the 'init' function of the controller.

Variable 'p' is of type 'Position'. This is defined by an enum containing 'left' and 'right'; this should go with the other enum classes.

Implementing the obstacle subscriber was a challenge; in the end, the message type was set to `int8`, and the callback reads 0 as left and 1 as right. This can be changed later if needed.

## 5 Implementation

The language around timed elements in the guidelines is not very specific. For example, it does not explicitly tell you how to define a timer or a clock.

The guidelines do not have a way of dealing with composite states. One way of doing this would be to have an enum of the outer states (any state not in another) and an enum of the inner states (any state in another). There would be a match case statement as normal, but any state with states inside will also contain a match case statement. This also requires splitting the action tracking into two, one for each inner and outer.

To allow for the node to be run for multiple robots, you can declare parameters and then get their values back from the node. This allows for different robots to be run in different namespaces and start in different positions (Listing 5.14).

Listing 5.14: Parameters for multiple robots.

```
1 self.declare_parameter('robot_id', 'robot1')
2 self.declare_parameter('x', 0.0)
3 self.declare_parameter('y', 0.0)
4 self.declare_parameter('yaw', 0.0)
5 self.robot_id = self.get_parameter('robot_id').value
6 self.x = self.get_parameter('x').get_parameter_value().
    double_value
7 self.y = self.get_parameter('y').get_parameter_value().
    double_value
8 self.yaw = self.get_parameter('yaw').get_parameter_value().
    double_value
```

Full code for the Movement controller can be found in Listing B.3. At this stage, there is little point in implementing the Communication controller. It provides very little that hasn't already been discussed, requires a lot of assumptions to be made, and there isn't much room left in the report.

## 5.5 Final Guidelines

The goal of both implementations was to create a set of guidelines for implementation. These guidelines are the results of the project. This section will only go through any differences from the first set of guidelines; the complete set of edited guidelines is in Appendix A.

- 1.1 For each controller in the model, start with a sensibly named file containing the initial code (Listing 5.13).

### 2 Constants and Variables

## 5 Implementation

+2.3 Variables should be defined in the '`__init__`' function.

+2.4 Try to set variables to what seems like the default.

+3.2d If you can't figure out a message type, set it to something that does roughly the same job and return later to fix it if needed.

+4.3d Timers are written in the form "`self.timer = self.create_timer(self.timer_period, self.function)`".

4.4b The clock is an integer that starts at 0; it will be incremented every time the control loop is pulsed by the timer.

### +7 Composite States

+7.1 Create an enum of the "outer" states, i.e. those not in others.

+7.2 Create an enum of the "inner" states, i.e. those in others.

+7.3 In addition to the existing match case statement, each state that contains other states will also have a match case statement.

+7.4 You may want to add the additional cases to redirect to the initial state of the composite state, just in case.

+7.5 If using the action stages, two are needed: '`self.outer_action`' and '`self.inner_action`'.

+7.6 The transition condition for an outer state should be after any inner state behaviour to avoid restarting the inner state machine.

+7.7 Final junctions mean that the state machine is finished.

### +8 Multiple Robots

+8.1 To allow your files to have configurable parameters, declare the parameter, then get it.

+8.2 For example, you could make the namespace configurable by writing "`self.declare_parameter('robot_id', 'robot1')`" and "`self.robot_id = self.get_parameter('robot_id').value`".

+8.3 Configurable parameters are important if you want to use the same file to run multiple nodes.

# 6 Evaluation

## 6.1 Case Studies

The implementation of the SRanger case study created as part of this project is very accurate to the original RoboChart model. It contains all the same states, behaviour and attributes. When simulated, the robot lasted more than 10 minutes without crashing into anything before the simulation was terminated (Figure 6.1).

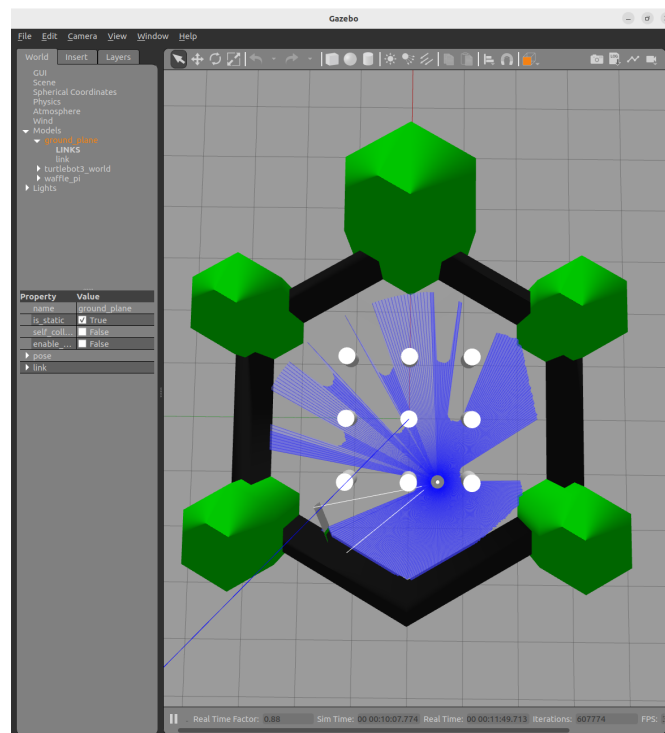


Figure 6.1: SRanger >10 mins runtime.

One change that could be considered for this code is adding an initial state. This was included as a suggestion in the initial guidelines, but was thought of after the implementation was complete.

A consideration with this implementation is that obstacle detection is not



included. The model requires the robotic platform to inform it of an obstacle. This means that any interface allowing the code to run on a different robotic platform needs to have object detection capabilities. The object detection used in the simulation was passable, but may not be enough for every scenario.

The implementation of the Alpha Algorithm Movement controller was done entirely with the knowledge in the final set of guidelines. While not simulated due to missing elements and many assumptions needing to be made, it remains a solid implementation and implementing it allowed some key points to be added to the guidelines. Outside of the obstacle interface, the implementation seems to be a faithful recreation of the RoboChart model.

## 6.2 Qualitative Analysis of Results

This project has resulted in the creation of 73 guidelines for translation from RoboChart to ROS. The guidelines are readable and well laid out, containing sections for different key concepts, allowing for easy reference. The guidelines cover most of the concepts of the RoboChart language that would be regularly seen.

During the implementation of the Alpha Algorithm, the guidelines were followed carefully. Anytime information was needed from another source, a guideline was added in, and anytime a new concept was introduced, new guidelines with as in-depth an exploration as possible were added. With the final guidelines and a bit of common sense, the implementation should be able to be recreated by someone else. The guidelines were found to be extremely helpful while trying to implement something, almost acting as a quick reference guide at times, and they were easy to follow.

Accompanying the guidelines is a figure containing most, if not all, of the icons used in RoboChart notation and what they mean. To my knowledge, something like this does not exist. It is a very useful addition and makes everything a bit more comprehensible.

Of the fifteen things listed as "Core Language" in the well-formedness conditions of the RoboChart manual, only 3 are not directly addressed in the guidelines. They are all very conceptual and are: junctions, communications and expressions.

## 6.3 Success Criteria Review

Seven success criteria were laid out in the Methodology and Design section.

The first was **"SRanger RoboChart model implemented"**. This is a success, the SRanger model was successfully implemented.

The second was **"SRanger runs in a ROS environment"**. This is also a success, the SRanger implementation was able to run in a ROS environment. It ran successfully for over ten minutes and would've kept going if it weren't terminated.

The third was **"Initial Guidelines created"**. A set of initial guidelines was created around the process of implementing SRanger, which is another success.

The fourth was **"Alpha Algorithm RoboChart model implemented"**. This is the first one with an issue. The more complex of the two controllers from the model was implemented, though it was never simulated. This is mostly a success, but not entirely.

The fifth was **"Guidelines updated to final version"**. This is a success! The implementation of Alpha Algorithm's Movement controller allowed for the creation of 14 new guidelines and 2 edits to existing ones.

The sixth was **"Guidelines are coherent and easy to follow"**. This one is a bit more subjective, however, I do believe it to be true. This counts as a success.

The seventh, final success criterion was **"Guidelines cover a good proportion of the RoboChart language"**. This can be seen as another success.

## 6.4 Conclusion

To conclude, the aim of this project to create a set of guidelines for implementing RoboChart models in ROS has been achieved. All of the success criteria set out were at least mostly completed. A RoboChart model was successfully translated, implemented, and simulated, with another being mostly implemented. Both of these case studies combined to pave the way for a solid set of guidelines.

# A Full Guidelines

## 1. Set Up

- 1.1 For each controller in the model, start with a sensibly named file containing the initial code (Listing 5.13).
- 1.2 Replace instances of `Controller_Name` with the name of your controller, keeping the original capitalisation (lines 16, 18, 43).

## 2. Constants and Variables

- 2.1 Constants should be defined below the imports.
- 2.2 Most constants are not assigned values in models; set them to something and tune them later.
- 2.3 Variables should be defined in the `'__init__'` function.
- 2.4 Try to set variables to what seems like the default.

## 3. Interfaces

- 3.1 Interfaces should be defined in the `'__init__'` function.
- 3.2 Interfaces require a message type.
  - 3.2a *When sending linear and/or angular velocities, this will be 'Twist' from 'geometry\_msgs.msg'.*
  - 3.2b *If all that needs sending is a signal, this will be 'Empty' from 'std\_msgs.msg'.*
  - 3.2c *In certain cases, other message types may be required.*
  - 3.2d *If you can't figure out a message type, set it to something that does roughly the same job and return later to fix it if needed.*
- 3.3 Interfaces require a topic to use.

## A Full Guidelines

*3.3a This must start with a '/*

*3.3b The topic will have the name of the operation/event from the model unless required otherwise.*

3.4 Interfaces require a buffer size.

*3.4a Usually, a buffer size of 10 is more than sufficient.*

3.5 Subscribers require a reference to a callback.

*3.5a The callback will be a function in the controller module.*

*3.5b The callback will define what happens when a message is received.*

*3.5c The callback needs 'self' as a parameter.*

*3.5d The callback needs 'msg' as a parameter.*

*3.5e If the message is empty, the callback may toggle a boolean, allowing processing to happen elsewhere. Ensure this is toggled back at an appropriate point.*

*3.5f If the message has contents, the callback may facilitate storing or making decisions based on them.*

3.6 Subscribers are written in the form "self.subscription\_name = self.create\_subscription(Message\_type, '/topic\_name', self.callback\_name, int buffer\_size)".

3.7 Publishers are written in the form "self.publisher\_name = self.create\_publisher(Message\_type, '/topic\_name', int buffer\_size)".

3.8 When publishing a message is required, include the line "self.publisher\_name.publish()".

*3.8a You must create a 'msg' variable that will be passed as a parameter (even if it's an empty message).*

*3.8b If the message has contents, you must populate it; an example of a Twist 'msg' creation is in Listing A.1. Twist components must be floats.*

Listing A.1: Twist 'msg'.

```
1 msg = Twist()  
2 msg.linear.x = LINEAR_VELOCITY  
3 msg.angular.z = ANGULAR_VELOCITY
```

#### 4. Timed elements

4.1 If there is a clock defined in your model, you will need to use timed elements.

*4.1a All of these elements are part of the class of your controller.*

4.2 RoboChart models are written in time units.

*4.2a This should be defined and can be changed later.*

*4.2b 1 time unit to 1 second is usually a good starting point.*

*4.2c All timed elements should be written in time units to allow easy changes later.*

4.3 Timed elements require a timer.

*4.3a Timers provide a consistent pulse activating something, in this case, the control loop.*

*4.3b Timers require a timer period, which will decide the frequency. e.g. timer period of 0.1 = 10Hz.*

*4.3c Timers require a function to execute, in this case, the control loop.*

*4.3d Timers are written in the form "self.timer = self.create\_timer(self.timer\_period, self.function)".*

4.4 Timed elements require a clock.

*4.4a This will share a name with the clock in the model.*

*4.4b The clock is an integer that starts at 0; it will be incremented every time the control loop is pulsed by the timer.*

*4.4c A '#' preceding the clock's name in a model means the clock is reset at that point; this can be done by setting the clock to 0.*

*4.4d In the model, since(clock\_name) asks for the time since the clock was last reset, i.e. the time on the clock. This is done with the calculation "self.timer\_period \* self.clock\_name / self.time\_unit". It may be useful to make this a function.*

4.5 Timed elements allow you to implement waits.

## A Full Guidelines

4.5a *If the model includes waits, create a variable in the controller's class for saving the time on the clock.*

4.5b *First, save the time on the clock (Guideline 4.4d) before you transition to another action/state that requires a wait.*

4.5c *Then create an if statement where the wait is that takes the saved time away from the current clock time and compares it to the number of time units the wait is for. i.e. "current - saved >= x(time\_unit)".*

## 5. State Machines

5.1 Create a list of all the states in the state machine you are implementing.

5.1a *To match more closely with the RoboChart model, include the initial state.*

5.2 Each state should be added to the Enum class 'State', with the initial state being set to 0 and every subsequent state added being 1 higher.

5.3 Each state should be added to the match case statement.

5.4 Ensure 'self.state' is set to either the initial state if included, or the state it points to.

5.5 If included, the initial state should reset any clocks in use and then transition to the next state.

## 6. State Behaviour

6.1 States have three action types: 'entry', 'during', and 'exit'.

6.1a *While not 'during' actions, transition conditions take place at the same time and so should be implemented in the 'during' section.*

6.1b *If a state has only one transition out of it, it may be convenient to put the consequences of the transition condition in the 'exit' section, despite it not technically being an 'exit' action.*

6.2 Most behaviour will be either using previously set up clocks, interfaces, etc., performing calculations, or both.

6.3 Change state using "self.state = State.STATE\_NAME".

- 6.4 Change action using "self.action = Action.ACTION\_NAME".
- 6.5 Before changing states, ensure 'self.action' is also set to 'State.ENTRY'.

## **7. Composite States**

- 7.1 Create an enum of the "outer" states, i.e. those not in others.
- 7.2 Create an enum of the "inner" states, i.e. those in others.
- 7.3 In addition to the existing match case statement, each state that contains other states will also have a match case statement.
- 7.4 You may want to add the additional cases to redirect to the initial state of the composite state, just in case.
- 7.5 If using the action stages, two are needed: 'self.outer\_action' and 'self.inner\_action'.
- 7.6 The transition condition for an outer state should be after any inner state behaviour to avoid restarting the inner state machine.
- 7.7 Final junctions mean that the state machine is finished.

## **8. Multiple Robots**

- 8.1 To allow your files to have configurable parameters, declare the parameter, then get it.
- 8.2 For example, you could make the namespace configurable by writing "self.declare\_parameter('robot\_id', 'robot1')" and "self.robot\_id = self.get\_parameter('robot\_id').value".
- 8.3 Configurable parameters are important if you want to use the same file to run multiple nodes.

## **9. Controllers should send a stop command of some kind when the node is destroyed.**

## B Code Appendix

Listing B.1: SRanger Code.

```
1 import sys
2 import math
3 import rclpy
4 from rclpy.node import Node
5 from rclpy.signals import SignalHandlerOptions
6 from rclpy.executors import ExternalShutdownException,
    MultiThreadedExecutor
7 from geometry_msgs.msg import Twist
8 from std_msgs.msg import Empty
9 from enum import Enum
10
11 LINEAR_VELOCITY = 0.3 #meters per second
12 ANGULAR_VELOCITY = 1.0 #radians per second
13 PI = math.pi
14
15 class State(Enum): #FSM states
16     MOVING = 0
17     TURNING = 1
18
19 class Action(Enum): #State actions
20     ENTRY = 0
21     DURING = 1
22     EXIT = 2
23
24 class Movement(Node):
25     def __init__(self):
26         super().__init__('robot_controller')
27
28         self.state = State.MOVING
29         self.action = Action.ENTRY
30         self.obstacle_found = False #Has an obstacle been
            detected?
31         self.waiting = 0.1 #Ignores first clock increase
32
33         #Clock
34         self.clock_MBC = 0
35
36         # Periodic callback for executing FSM
37         self.timer_period = 0.1 # 100 milliseconds = 10 Hz for
            the control loop
38         self.time_unit = 1 # 1 second is the value chosen for
            one 'time unit'
39         self.timer = self.create_timer(self.timer_period, self.
```



```

        control_loop)
40
41    #Publishers
42    self.move_publisher = self.create_publisher(Twist, '/
        move', 10)
43    self.stop_publisher = self.create_publisher(Empty, '/
        stop', 10)
44
45    #Subscriber
46    self.obstacle_subscriber = self.create_subscription(
47        Empty,
48        '/obstacle',
49        self.obstacle_callback,
50        10)
51
52    def since_clock_MBC(self): #Get current time since clock
        reset
53        return self.timer_period * self.clock_MBC / self.
            time_unit
54
55    def obstacle_callback(self, msg): #Subscriber callback
56        self.obstacle_found = True
57
58    def control_loop(self):
59        self.clock_MBC += 1 #Increment clock
60        match self.state:
61            case State.MOVING:
62                match self.action:
63                    case Action.ENTRY:
64                        #move(lv, 0)
65                        msg = Twist()
66                        msg.linear.x = LINEAR_VELOCITY
67                        msg.angular.z = 0.0
68                        self.move_publisher.publish(msg)
69                        #wait(1)
70                        if self.since_clock_MBC() - self.waiting
                            >= self.time_unit:
71                            self.action = Action.DURING
72                    case Action.DURING:
73                        if self.obstacle_found == True:
74                            self.clock_MBC = 0 #'#MBC'
75                            self.action = Action.EXIT
76                    case Action.EXIT:
77                        #stop()
78                        msg = Empty()
79                        self.stop_publisher.publish(msg)
80                        #wait(1)
81                        if self.since_clock_MBC() >= self.
                            time_unit:
82                            self.waiting = self.since_clock_MBC
                                ()
83                            self.action = Action.ENTRY
84                            self.state = State.TURNING
85                    case _:
86                        pass

```

## B Code Appendix

```
87         case State.TURNING:
88             match self.action:
89                 case Action.ENTRY:
90                     #move(0, av)
91                     msg = Twist()
92                     msg.linear.x = 0.0
93                     msg.angular.z = ANGULAR_VELOCITY
94                     self.move_publisher.publish(msg)
95                     #wait(1)
96                     if self.since_clock_MBC() - self.waiting
97                         >= self.time_unit:
98                         self.action = Action.DURING
99                 case Action.DURING:
100                     #since(MBC)>=PI/av
101                     if self.since_clock_MBC() >= PI/
102                         ANGULAR_VELOCITY:
103                         self.waiting = self.since_clock_MBC
104                         ()
105                         self.obstacle_found = False
106                         self.action = Action.ENTRY
107                         self.state = State.MOVING
108             case _:
109                 pass
110         case _:
111             pass
112
113     def destroy_node(self):
114         msg = Empty()
115         self.stop_publisher.publish(msg)
116         super().destroy_node()
117
118 def main(args=None):
119     rclpy.init(args = args, signal_handler_options =
120         SignalHandlerOptions.NO)
121
122     node = Movement()
123
124     executor = MultiThreadedExecutor()
125     executor.add_node(node)
126
127     try:
128         executor.spin()
129     except KeyboardInterrupt:
130         pass
131     except ExternalShutdownException:
132         sys.exit(1)
133     finally:
134         node.destroy_node()
135         rclpy.try_shutdown()
136
137 if __name__ == '__main__':
138     main()
```

Listing B.2: Interface Code.

```

1 import sys
2 import rclpy
3 from rclpy.node import Node
4 from rclpy.signals import SignalHandlerOptions
5 from rclpy.executors import ExternalShutdownException,
   MultiThreadedExecutor
6 from geometry_msgs.msg import Twist
7 from std_msgs.msg import Empty
8 from rclpy.qos import QoSProfile, QoSPredefinedProfiles
9 from sensor_msgs.msg import LaserScan
10
11 SCAN_THRESHOLD = 0.5 # Metres per second
12
13 class Interface(Node):
14     def __init__(self):
15         super().__init__('interface')
16
17         self.move_subscription = self.create_subscription(Twist,
18                                                         '/move', self.move_callback, 10)
19         self.stop_subscription = self.create_subscription(Twist,
20                                                         '/stop', self.stop_callback, 10)
21         self.scan_subscriber = self.create_subscription(
22             LaserScan,
23             'scan',
24             self.scan_callback,
25             QoSPredefinedProfiles.SENSOR_DATA.value)
26
27         self.cmd_vel_publisher = self.create_publisher(Twist, '/cmd_vel', 10)
28         self.obstacle_publisher = self.create_publisher(Empty, '/obstacle', 10)
29
30     def move_callback(self, msg):
31         self.cmd_vel_publisher.publish(msg)
32
33     def stop_callback(self, msg):
34         msg2 = Twist()
35         self.cmd_vel_publisher.publish(msg2)
36
37     def scan_callback(self, msg):
38         front_ranges = msg.ranges[331:359] + msg.ranges[0:30] #
39         # 30 to 331 degrees (30 to -30 degrees)
40
41         if min(front_ranges) < SCAN_THRESHOLD:
42             msg = Empty()
43             self.obstacle_publisher.publish(msg)
44
45     def destroy_node(self):
46         msg = Twist()
47         self.cmd_vel_publisher.publish(msg)
48         super().destroy_node()
49
50 def main(args=None):

```

## B Code Appendix

```
48     rclpy.init(args = args, signal_handler_options =
        SignalHandlerOptions.NO)
49     node = Interface()
50     executor = MultiThreadedExecutor()
51     executor.add_node(node)
52     try:
53         executor.spin()
54     except KeyboardInterrupt:
55         pass
56     except ExternalShutdownException:
57         sys.exit(1)
58     finally:
59         node.destroy_node()
60         rclpy.try_shutdown()
61
62 if __name__ == '__main__':
63     main()
```

Listing B.3: Alpha Algorithm Movement Controller

```
1 import sys
2 import rclpy
3 from rclpy.node import Node
4 from rclpy.signal import SignalHandlerOptions
5 from rclpy.executors import ExternalShutdownException,
    MultiThreadedExecutor
6 from geometry_msgs.msg import Twist
7 from std_msgs.msg import Int8, Bool
8 from enum import Enum
9 import math
10 import random
11
12 LINEAR_VELOCITY = 0.3
13 ANGULAR_VELOCITY = 1.0
14 MB = 361.0
15 ALPHA = 3
16
17 class Outer_State(Enum):
18     INITIAL = 0
19     MOVE_AND_AVOID = 1
20     TURNING = 2
21
22 class Inner_State(Enum):
23     INITIAL = 0
24     MOVE = 1
25     AVOID = 2
26     TURN = 3
27     RAND_TURN = 4
28
29 class Action(Enum):
30     ENTRY = 0
31     DURING = 1
32     EXIT = 2
33
34 class Position(Enum):
```

```
35     left = 0
36     right = 1
37
38 class Movement(Node):
39     def __init__(self):
40         super().__init__('movement')
41
42         self.outer_state = Outer_State.INITIAL
43         self.inner_state = Inner_State.INITIAL
44         self.outer_action = Action.ENTRY
45         self.inner_action = Action.ENTRY
46
47         self.n = int(0)
48         self.p = Position.left
49         self.turned = False
50         self.obstacle_found = False
51         self.time_unit = 0.1
52         self.timer_period = 0.1
53         self.clock_MBC = 0
54         self.waiting = 0
55         self.wait = 0
56
57         self.declare_parameter('robot_id', 'robot1')
58         self.declare_parameter('x', 0.0)
59         self.declare_parameter('y', 0.0)
60         self.declare_parameter('yaw', 0.0)
61         self.robot_id = self.get_parameter('robot_id').value
62         self.x = self.get_parameter('x').get_parameter_value().
            double_value
63         self.y = self.get_parameter('y').get_parameter_value().
            double_value
64         self.yaw = self.get_parameter('yaw').get_parameter_value
            ().double_value
65
66         self.timer = self.create_timer(self.timer_period, self.
            control_loop)
67
68         self.move_publisher = self.create_publisher(Twist, '/'
            move', 10)
69
70         #Message type needs changing - maybe
71         self.obstacle_subscriber = self.create_subscriber(Int8,
            '/obstacle', self.obstacle_callback, 10)
72         self.neighbours_subscriber = self.create_subscriber(Int8
            , '/neighbours', self.neighbours_callback, 10)
73
74         def since_clock_MBC(self): #Get current time since clock
            reset
75             return self.timer_period * self.clock_MBC / self.
                time_unit
76
77         def obstacle_callback(self, msg):
78             if msg == 0:
79                 self.p = Position.left
80             if msg == 1:
```

## B Code Appendix

```
81         self.p = Position.right
82         self.obstacle_found = True
83
84     def neighbours_callback(self, msg):
85         self.n = int(msg)
86
87     def control_loop(self):
88         self.clock_MBC += 1
89         match self.outer_state:
90             case Outer_State.INITIAL:
91                 self.clock_MBC = 0
92                 self.obstacle_found = False
93                 self.outer_state = Outer_State.MOVE_AND_AVOID
94             case Outer_State.MOVE_AND_AVOID:
95                 match self.outer_action:
96                     case Action.ENTRY:
97                         self.outer_action = Action.DURING
98                     case Action.DURING:
99                         match self.inner_state:
100                             case Inner_State.INITIAL:
101                                 self.inner_state = Inner_State.
102                                     MOVE
103                             case Inner_State.MOVE:
104                                 match self.inner_action:
105                                     case Action.ENTRY:
106                                         msg = Twist()
107                                         msg.linear.x =
108                                             LINEAR_VELOCITY
109                                         self.move_publisher.
110                                             publish(msg)
111                                         self.inner_action =
112                                             Action.DURING
113                                     case Action.DURING:
114                                         if self.obstacle_found
115                                             == True and self.
116                                                 since_clock_MBC() < MB
117                                                 -360/ANGULAR_VELOCITY
118                                                 :
119                                             self.inner_action =
120                                                 Action.ENTRY
121                                             self.inner_state =
122                                                 Inner_State.AVOID
123                                     case _:
124                                         pass
125             case Inner_State.AVOID:
126                 match self.inner_action:
127                     case Action.ENTRY:
128                         msg = Twist()
129                         if self.p == Position.
130                             left:
131                             msg.angular.z =
132                                 ANGULAR_VELOCITY
133                         else:
134                             msg.angular.z = -
135                                 ANGULAR_VELOCITY
```

## B Code Appendix

```
123         self.move_publisher.  
            publish(msg)  
124         self.waiting = self.  
            since_clock_MBC()  
125         self.wait = math.floor(  
            random.uniform(0, 1)  
            *360/ANGULAR_VELOCITY  
            )  
126         self.inner_action =  
            Action.DURING  
127         case Action.DURING:  
128             if self.since_clock_MBC  
                () - self.waiting >=  
                self.wait:  
129                 self.obstacle_found  
                    = False  
130                 self.inner_action =  
                    Action.ENTRY  
131                 self.inner_state =  
                    Inner_State.MOVE  
132             case _:  
133                 pass  
134         case _:  
135             self.inner_state = Inner_State.  
                INITIAL  
136             self.inner_action = Action.ENTRY  
137         if self.since_clock_MBC() >= MB:  
138             self.since_clock_MBC() = 0  
139             self.inner_action = Action.ENTRY  
140             self.outer_action = Action.ENTRY  
141             self.inner_state = Inner_State.  
                INITIAL  
142             self.outer_state = Outer_State.  
                TURNING  
143         case Outer_State.TURNING:  
144             match self.outer_action:  
145                 case Action.ENTRY:  
146                     self.turned = False  
147                 case Action.DURING:  
148                     match self.inner_state:  
149                         case Inner_State.INITIAL:  
150                             self.inner_action = Action.ENTRY  
151                             if self.n < ALPHA:  
152                                 self.inner_state =  
                                    Inner_State.TURN  
153                             elif self.n >= ALPHA:  
154                                 self.inner_state =  
                                    Inner_State.RAND_TURN  
155                         case Inner_State.TURN:  
156                             match self.inner_action:  
157                                 case Action.ENTRY:  
158                                     msg = Twist()  
159                                     msg.angular.z =  
                                        ANGULAR_VELOCITY  
160                                     self.move_publisher.
```

## B Code Appendix

```
161         publish(msg)
162         self.waiting = self.
163             since_clock_MBC()
164         self.inner_action =
165             Action.DURING
166     case Action.DURING:
167         if self.since_clock_MBC
168             () - self.waiting >=
169             math.floor(180/
170             ANGULAR_VELOCITY):
171             self.turned = True
172     case _:
173         pass
174     case Inner_State.RAND_TURN:
175         match self.inner_action:
176             case Action.ENTRY:
177                 msg = Twist()
178                 msg.angular.z =
179                     ANGULAR_VELOCITY
180                 self.move_publisher(msg)
181                 self.waiting = self.
182                     since_clock_MBC()
183                 self.wait = math.floor(
184                     random.uniform(0, 1)
185                     *360/ANGULAR_VELOCITY
186                 )
187                 self.inner_action =
188                     Action.DURING
189             case Action.DURING:
190                 if self.since_clock_MBC
191                     () - self.waiting >=
192                     self.wait:
193                     self.turned = True
194             case _:
195                 pass
196     case _:
197         self.inner_state = Inner_State.
198             INITIAL
199         self.inner_action = Action.ENTRY
200     if self.turned == True:
201         self.inner_action = Action.ENTRY
202         self.outer_action = Action.ENTRY
203         self.inner_state = Inner_State.
204             INITIAL
205         self.outer_state = Outer_State.
206             MOVE_AND_AVOID
207
208     case _:
209         pass
210
211     def destroy_node(self):
212         super().destroy_node()
213
214 def main(args=None):
215     rclpy.init(args = args, signal_handler_options =
```



## *B Code Appendix*

```
SignalHandlerOptions.NO)
199 node = Movement()
200 executor = MultiThreadedExecutor()
201 executor.add_node(node)
202 try:
203     executor.spin()
204 except KeyboardInterrupt:
205     pass
206 except ExternalShutdownException:
207     sys.exit(1)
208 finally:
209     node.destroy_node()
210     rclpy.try_shutdown()
211
212 if __name__ == '__main__':
213     main()
```

# Bibliography

- [1] A. Miyazawa, P. Ribeiro, W. Li, A. Cavalcanti, J. Timmis and J. Woodcock, 'Robochart: Modelling and verification of the functional behaviour of robotic applications,' *Software and Systems Modelling*, vol. 18, no. 5, pp. 3097–3149, Oct. 2019.
- [2] Object Management Group, *Unified modeling language*, <https://www.omg.org/spec/UML/2.5.1>, Dec. 2015.
- [3] A. Cavalcanti, A. Sampaio, A. Miyazawa *et al.*, 'Verified simulation for robotics,' *Science of Computer Programming*, vol. 174, pp. 1–37, 2019.
- [4] A. Miyazawa, P. Ribeiro, A. Cavalcanti, M. Windsor, J. Baxter and K. Ye, *SRanger*, [https://robostar.cs.york.ac.uk/case\\_studies/sranger/index.html](https://robostar.cs.york.ac.uk/case_studies/sranger/index.html), Accessed: 2025-04-23.
- [5] S. Macenski, T. Foote, B. Gerkey, C. Lalancette and W. Woodall, 'Robot Operating System 2: Design, architecture, and uses in the wild,' *Science Robotics*, vol. 7, no. 66, eabm6074, May 2022.
- [6] Open Robotics, *Understanding topics — ROS 2 Documentation: Humble documentation*, <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html>, Accessed: 2025-04-25.
- [7] Object Management Group, *System modeling language*, <https://www.omg.org/spec/SysML/2.0/Beta2>, Apr. 2024.
- [8] A. Nordmann, N. Hochgeschwender, D. Wigand and S. Wrede, 'A survey on domain-specific modeling and languages in robotics,' *Journal of Software Engineering for Robotics*, vol. 7, pp. 75–99, Jul. 2016.
- [9] S. Dhouib, S. Kchir, S. Stinckwich, T. Ziadi and M. Ziane, 'Robotml, a domain-specific language to design, simulate and deploy robotic applications,' in *Simulation, Modeling, and Programming for Autonomous Robots*, Springer, Berlin, Heidelberg, 2012, pp. 149–160.
- [10] M. Foughali, B. Berthomieu, S. Dal Zilio, F. Ingrand and A. Mallet, 'Model checking real-time properties on the functional layer of autonomous robots,' in *Formal Methods and Software Engineering*, Springer International Publishing, 2016, pp. 383–399.

## Bibliography

- [11] A. Elkady and T. Sobh, 'Robotics middleware: A comprehensive literature survey and attribute-based bibliography,' *Journal of Robotics*, vol. 2012, no. 1, p. 959 013, 2012.
- [12] Open Robotics, *Upcoming ROS 1 End of Life*, <https://www.ros.org/blog/noetic-eol/>, Accessed: 2025-04-24.
- [13] Open Robotics, *Distributions – ros 2 documentation: Rolling documentation*, <https://docs.ros.org/en/rolling/Releases.html>, Accessed: 2025-04-25.
- [14] N. Shaukat, S. Dubey, B. Kaddouh *et al.*, 'Trustworthy ros software architecture for autonomous drones missions: From robochart modelling to ros implementation,' in *20th IEEE/ASME International Conference on Mechatronic and Embedded Systems and Applications (MESA)*, 2024, pp. 1–7.
- [15] M. Santos, M. C. Filho and A. Sampaio, 'A model-based approach to the development and verification of robotic systems for competitions,' in *Latin American Robotics Symposium (LARS), 2023 Brazilian Symposium on Robotics (SBR), and 2023 Workshop on Robotics in Education (WRE)*, 2023, pp. 236–241.
- [16] L. Batten, 'Implementing robochart models in ros,' Student Project, University of York, 2023.
- [17] RoboStar, *Case studies*, [https://robostar.cs.york.ac.uk/case\\_studies/](https://robostar.cs.york.ac.uk/case_studies/), Accessed: 2025-04-23.
- [18] A. Miyazawa, P. Ribeiro and A. Cavalcanti, *Alpha Algorithm*, [https://robostar.cs.york.ac.uk/case\\_studies/alpha\\_algorithm/alpha\\_algorithm.html](https://robostar.cs.york.ac.uk/case_studies/alpha_algorithm/alpha_algorithm.html), Accessed: 2025-04-23.
- [19] A. Miyazawa, A. Cavalcanti, P. Ribeiro *et al.*, 'RoboChart Reference Manual,' University of York, Technical report, Feb. 2016. [Online]. Available: <https://www.cs.york.ac.uk/circus/publications/techreports/reports/robochart-reference.pdf>.
- [20] Open Robotics, *Configuring environment*, <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Configuring-ROS2-Environment.html>, Accessed: 2025-05-01.
- [21] N. Koenig and A. Howard, 'Design and use paradigms for gazebo, an open-source multi-robot simulator,' in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, vol. 3, 2004, pp. 2149–2154.
- [22] Open Robotics, *Developing a ros 2 package*, <https://docs.ros.org/en/humble/How-To-Guides/Developing-a-ROS-2-Package.html>, Accessed: 2025-05-01.